

ELAW 프로젝트 — DB 설계 문서

프로젝트 ELAW (취업 학습 플랫폼)

DB 엔진 Supabase PostgreSQL (운영) / SQLite3 (로컬 풀백)

ORM Django 6.0.4

총 테이블 32개

작성일 2026-04-27

최종 수정 2026-06-05

목차

- 1. 전체 구조 개요
- 2. 레이어 1 — 기본 도메인 (16개 테이블)
- 3. 레이어 2 — 데이터셋 (5개 테이블)
- 4. 레이어 3 — 문제·퀴즈 (7개 테이블)
- 5. 레이어 4 — 추천·갭·포트폴리오 (4개 테이블)
- 6. 테이블 관계도 (ERD 요약)
- 7. 데이터 흐름
- 8. 마이그레이션 히스토리
- 9. 관리 커맨드
- 10. DB 조회 API
- 11. 환경 설정
- 12. 현재 데이터 현황

1. 전체 구조 개요

ELAW DB는 4개 레이어로 구성됩니다. 각 레이어는 독립적인 모델 파일로 관리되며, `models_register.py` 를 통해 Django ORM에 일괄 등록됩니다.

레이어 1 기본 도메인 — `models.py`

사용자, 기업, 학습 목표, 커리큘럼, 풀이 이력, 채용, 포트폴리오, 게시판(첨부·조회·좋아요·댓글 포함) 등 플랫폼 핵심 16개 테이블

레이어 2 데이터셋 — `models_dataset.py`

HuggingFace 이력서·채용공고 원본 데이터셋 저장 및 AI 파싱 결과 5개 테이블

레이어 3 문제 — `models_problems.py`

직군별 6,000개 문제, 클러스터, 선수과목 그래프, 학습 경로 메타 + 진단 퀴즈·추천 세션 7개 테이블

레이어 4 추천·갭·포트폴리오 — `models_new.py`

스킬 갭 분석, 문제 추천, 포트폴리오 버전 관리, RLHF 피드백 4개 테이블

파일 구조

```
backend/core/
├── models.py           # 레이어 1 (기본 도메인 16개 모델 - 게시판 확장 포함)
├── models_dataset.py   # 레이어 2 (데이터셋 5개 모델)
├── models_problems.py  # 레이어 3 (문제 5개 + 퀴즈 세션 2개 모델)
├── models_new.py       # 레이어 4 (추천·갭·포트폴리오 4개 모델)
├── models_register.py  # 전체 모델 Django 앱 등록 허브
├── migrations/
│   ├── 0001_initial.py      # 기본 12개 테이블
│   ├── 0002_user_is_staff_... py # User 필드 추가
│   ├── 0003_dataset_tables.py # 데이터셋 5개 테이블
│   ├── 0004_merge_20260421.py  # 병합
│   ├── 0005_alter_datasetentry_... py # 문제 5개 테이블 추가
│   ├── 0006_portfoliosnapshot_... py # 추천·갭·포트폴리오 4개 테이블 추가
│   └── 0007_add_skillgap_status_... py # SkillGap.status, 추천 플랫폼에 elaw 추가
```

```
|   ├── 0008_add_quiz_recommend_sessions.py # 퀴즈·추천 세션 2개 테이블
|   ├── 0009_board_comments_likes_views.py # 댓글·좋아요·조회 3개 테이블
|   ├── 0010_post_attachments.py           # 게시물 첨부파일 테이블
|   └── 0011_alter_jobproblem_question_type.py # question_type 길이 확장
└─ management/commands/
    ├── load_dataset.py      # HuggingFace 데이터셋 적재
    ├── load_problems.py     # JobProblems + LearningPaths 적재
    ├── seed_all.py          # 테스트용 시드 데이터 생성
    ├── fill_tables.py       # 기초 데이터 채우기
    └── sync_platforms.py    # 백준·GitHub API 동기화 (ETL)
```

2. 레이어 1 — 기본 도메인 (16개 테이블)

실제 테이블명: 레이어 1 모델은 `db_table` 을 지정하지 않으므로 실제 테이블명은 Django 기본 규칙을 따릅니다 (예: `users` → `core_user` , `platform_links` → `core_platformlink` , `solve_history` → `core_solvehistory`). 본 문서의 표제는 논리명입니다. 레이어 2~4는 `db_table` 이 명시되어 표제가 곧 실제 테이블명입니다.

2-1. users (사용자)

`AUTH_USER_MODEL` 플랫폼 통합 사용자 계정. role로 학생/기업/관리자를 구분합니다.

필드	타입	제약	설명
<code>id</code>	<code>BigAutoField</code>	PK	자동 증가 기본키
<code>email</code>	<code>CharField(254)</code>	UNIQUE	로그인 ID
<code>password_hash</code>	<code>CharField(128)</code>	NOT NULL	bcrypt 해시
<code>name</code>	<code>CharField(100)</code>	NOT NULL	이름
<code>role</code>	<code>CharField(10)</code>	NOT NULL	student / company / admin
<code>phone</code>	<code>CharField(20)</code>	nullable	전화번호
<code>is_active</code>	<code>BooleanField</code>	default=True	계정 활성 여부
<code>is_staff</code>	<code>BooleanField</code>	default=False	Django admin 접근
<code>ai_consent</code>	<code>BooleanField</code>	default=False	AI 분석 동의
<code>privacy_consent</code>	<code>BooleanField</code>	default=False	개인정보 동의
<code>created_at</code>	<code>DateTimeField</code>	auto_now_add	
<code>updated_at</code>	<code>DateTimeField</code>	auto_now	

인덱스: `idx_users_role` `idx_users_created_at`

2-2. companies (기업)

User(role=company)와 1:1 연결되는 기업 프로필.

필드	타입	제약	설명
<code>id</code>	<code>BigAutoField</code>	PK	

필드	타입	제약	설명
<code>user_id</code>	<code>OneToOneField</code>	FK→users	
<code>name</code>	<code>CharField(200)</code>	NOT NULL	기업명
<code>industry</code>	<code>CharField(100)</code>	nullable	업종
<code>description</code>	<code>TextField</code>	nullable	기업 소개
<code>logo_url</code>	<code>CharField(500)</code>	nullable	
<code>website_url</code>	<code>CharField(500)</code>	nullable	
<code>is_approved</code>	<code>BooleanField</code>	default=False	관리자 승인 여부
<code>approved_at</code>	<code>DateTimeField</code>	nullable	
<code>created_at</code>	<code>DateTimeField</code>	auto_now_add	

인덱스: `idx_companies_approved`

2-3. platform_links (외부 플랫폼 연동)

백준·GitHub·프로그래머스 계정 연동 정보. (user, platform) 쌍이 유니크합니다.

필드	타입	제약	설명
<code>id</code>	<code>BigAutoField</code>	PK	
<code>user_id</code>	<code>ForeignKey</code>	FK→users	
<code>platform</code>	<code>CharField(20)</code>	NOT NULL	baekjoon / github / programmers
<code>external_id</code>	<code>CharField(100)</code>	NOT NULL	플랫폼 내 사용자 ID
<code>access_token</code>	<code>CharField(500)</code>	nullable	OAuth 토큰
<code>token_expires</code>	<code>DateTimeField</code>	nullable	토큰 만료일
<code>last_synced</code>	<code>DateTimeField</code>	nullable	마지막 동기화 시각
<code>is_active</code>	<code>BooleanField</code>	default=True	

유니크: (user, platform) 인덱스: `idx_platform_links_synced`

2-4. user_goals (학습·취업 목표)

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users	
goal_type	CharField(10)	NOT NULL	study / job
field	CharField(100)	nullable	예) 백엔드, AI/ML
job_role	CharField(100)	nullable	예) 백엔드 개발자
start_date	DateField	nullable	
end_date	DateField	nullable	
duration_weeks	PositiveSmallIntegerField	nullable	
is_active	BooleanField	default=True	현재 활성 목표

인덱스: idx_user_goals_active

2-5. curricula (AI 커리큘럼)

Gemini API로 생성된 주차별 학습 계획. goal당 N개 버전 관리됩니다.

필드	타입	제약	설명
id	BigAutoField	PK	
goal_id	ForeignKey	FK→user_goals	
user_id	ForeignKey	FK→users	
content_json	JSONField	NOT NULL	{total_weeks, field, job_role, weeks[{week, theme, tasks, recommended_problems, estimated_hours}]}
version	PositiveSmallIntegerField	default=1	버전 번호
is_active	BooleanField	default=True	현재 사용 중인 버전
generated_at	DateTimeField	auto_now_add	

인덱스: idx_curricula_user_active idx_curricula_goal

2-6. solve_history (외부 플랫폼 풀이 이력)

백준·프로그래머스에서 동기화된 풀이 기록. (user, platform, problem_id)가 유니크합니다.

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users	
platform	CharField(20)	NOT NULL	baekjoon / programmers / w3schools / custom(ELAW 자체)
problem_id	CharField(50)	NOT NULL	플랫폼 내 문제 번호
problem_title	CharField(300)	nullable	
status	CharField(10)	NOT NULL	solved / failed / partial
language	CharField(30)	nullable	사용 언어
algo_tags	JSONField	nullable	예) ["DP", "그래프"]
difficulty	CharField(30)	nullable	Bronze5, Silver3 등
time_spent_min	PositiveSmallIntegerField	nullable	풀이 소요 시간(분)
solved_at	DateTimeField	nullable	
source	CharField(20)	default='api'	api / crawl / manual

유니크: (user, platform, problem_id) 인덱스: idx_solve_user_solved_at idx_solve_platform idx_solve_language

2-7. learning_stats (학습 통계)

언어·알고리즘 태그별 정답률 집계. solve_history에서 자동 재집계됩니다.

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users	
stat_type	CharField(20)	NOT NULL	language / algo_tag
stat_key	CharField(100)	NOT NULL	예) "Python", "DP"
total_count	PositiveIntegerField	default=0	총 시도 수
solved_count	PositiveIntegerField	default=0	정답 수
failed_count	PositiveIntegerField	default=0	오답 수
correct_rate	FloatField	nullable	정답률 0~100
last_solved_at	DateTimeField	nullable	

유니크: (user, stat_type, stat_key) 인덱스: idx_stats_correct_rate

2-8. portfolios (포트폴리오)

AI 포트폴리오 생성은 비동기로 동작합니다: `POST /api/jobs/<id>/apply/` 가 즉시 placeholder 행을 만들고(202 응답), 백그라운드 스레드가 Ollama 추론(2~4분) 완료 후 `content_json.status` 를 `generating` → `done` | `error` 로 갱신합니다. 공고 대상 포트폴리오는 `content_json.target_posting_id` 메타데이터로 매칭됩니다.

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users	
title	CharField(200)	NOT NULL	
summary_text	TextField	nullable	요약 문장
content_json	JSONField	nullable	섹션별 구조화 내용
public_slug	CharField(50)	UNIQUE	공개 URL 식별자
pdf_path	CharField(500)	nullable	PDF 파일 경로
language	CharField(5)	default='ko'	ko / en
version	PositiveSmallIntegerField	default=1	
is_public	BooleanField	default=True	외부 공개 여부

인덱스: idx_portfolios_user_id idx_portfolios_public

2-9. job_postings (채용공고)

필드	타입	제약	설명
id	BigAutoField	PK	
company_id	ForeignKey	FK→companies	
title	CharField(300)	NOT NULL	공고 제목
description	TextField	nullable	
required_skills	JSONField	NOT NULL	필수 스킬 목록
preferred_skills	JSONField	nullable	우대 스킬 목록
job_role	CharField(100)	nullable	직무명
career_level	CharField(20)	nullable	new / junior / mid / senior / any

필드	타입	제약	설명
deadline	DateField	nullable	마감일
is_active	BooleanField	default=True	
view_count	PositiveIntegerField	default=0	

인덱스: `idx_job_postings_company` `idx_job_postings_role` `idx_job_postings_deadline`

2-10. matches (AI 매칭)

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users	
posting_id	ForeignKey	FK→job_postings	
match_score	FloatField	NOT NULL	매칭 점수 0~100
status	CharField(15)	NOT NULL	recommended / viewed / scrapped / applied / rejected
applied_at	DateTimeField	nullable	지원 시각

유니크: `(user, posting)` 인덱스: `idx_matches_user_score` `idx_matches_posting_status`

2-11. posts (게시판)

필드	타입	제약	설명
id	BigAutoField	PK	
author_id	ForeignKey	FK→users (RESTRICT)	공지·대화·이벤트는 admin, Q&A는 일반 사용자
category	CharField(10)	NOT NULL	notice / contest / event / qna
title	CharField(300)	NOT NULL	
content	TextField	NOT NULL	
is_pinned	BooleanField	default=False	상단 고정
view_count	PositiveIntegerField	default=0	계정당 1회 (post_views로 중복 방지)
like_count	PositiveIntegerField	default=0	좋아요 수 (post_likes 집계 캐시)

인덱스: `idx_posts_category_date` `idx_posts_pinned`

2-12. post_attachments (게시글 첨부파일) 신규

게시글당 N개 첨부. 허용 확장자: 이미지(jpg·png·gif·webp), 문서(pdf·doc·docx·txt·xlsx·pptx), zip. 파일은 `media/board/%Y/%m/` 에 저장됩니다.

필드	타입	제약	설명
id	BigAutoField	PK	
post_id	ForeignKey	FK→posts (CASCADE)	
file	FileField	NOT NULL	저장 경로 board/연/월/
original_name	CharField(255)	NOT NULL	업로드 시 원본 파일명
file_size	PositiveIntegerField	NOT NULL	바이트
uploaded_at	DateTimeField	auto_now_add	

2-13. post_views (게시글 조회 이력) 신규

조회수를 계정당 1회만 카운트하기 위한 중복 방지 테이블.

필드	타입	제약	설명
id	BigAutoField	PK	
post_id	ForeignKey	FK→posts (CASCADE)	
user_id	ForeignKey	FK→users (CASCADE)	
viewed_at	DateTimeField	auto_now_add	

유니크: `(post, user)`

2-14. post_likes (게시글 좋아요) 신규

계정당 1회 좋아요. posts.like_count는 이 테이블의 집계 캐시입니다.

필드	타입	제약	설명
id	BigAutoField	PK	
post_id	ForeignKey	FK→posts (CASCADE)	
user_id	ForeignKey	FK→users (CASCADE)	
created_at	DateTimeField	auto_now_add	

유니크: `(post, user)`

2-15. comments (게시글 댓글) 신규

필드	타입	제약	설명
id	BigAutoField	PK	
post_id	ForeignKey	FK→posts (CASCADE)	
author_id	ForeignKey	FK→users (CASCADE)	
content	TextField	NOT NULL	
created_at	DateTimeField	auto_now_add	기본 정렬 기준 (오래된 순)
updated_at	DateTimeField	auto_now	

2-16. ai_logs (AI API 호출 이력)

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	ForeignKey	FK→users, nullable	SYSTEM 호출이면 NULL
feature	CharField(30)	NOT NULL	curriculum / weakness / recommendation / portfolio / matching / problem_gen
prompt_tokens	PositiveIntegerField	nullable	
output_tokens	PositiveIntegerField	nullable	
latency_ms	PositiveIntegerField	nullable	응답 시간(ms)
status	CharField(10)	NOT NULL	success / error / timeout / cached
error_message	TextField	nullable	

인덱스: idx_ai_logs_user_feature idx_ai_logs_created_at idx_ai_logs_status

3. 레이어 2 — 데이터셋 (5개 테이블)

HuggingFace 데이터셋 `recuse/synthetic_resume_jd_raw_dataset` (2,640건) 을 저장·파싱합니다.
`load_dataset` 커맨드로 적재, Gemini API로 구조화 파싱합니다.

3-1. dataset_entries (원본 데이터셋)

필드	타입	제약	설명
<code>id</code>	<code>BigAutoField</code>	PK	
<code>dataset_id</code>	<code>UUIDField</code>	UNIQUE	HuggingFace curr_type_id
<code>curr_type_str</code>	<code>CharField(200)</code>	NOT NULL	예) "Software Engineer_Junior_Google"
<code>job_title</code>	<code>CharField(100)</code>	nullable	파싱된 직무명
<code>career_level</code>	<code>CharField(20)</code>	nullable	junior / mid / senior / unknown
<code>company_name</code>	<code>CharField(100)</code>	nullable	파싱된 기업명
<code>resume_raw</code>	<code>TextField</code>	NOT NULL	원본 이력서 텍스트
<code>jd_raw</code>	<code>TextField</code>	NOT NULL	원본 채용공고 텍스트
<code>is_parsed</code>	<code>BooleanField</code>	default=False	Gemini 파싱 완료 여부

3-2. dataset_resumes (구조화 이력서)

DatasetEntry와 1:1. Gemini API로 파싱된 이력서 구조화 데이터.

필드	타입	설명
<code>entry_id</code>	<code>OneToOneField</code>	FK→dataset_entries
<code>candidate_name / email / phone</code>	<code>CharField</code>	기본 인적사항
<code>linkedin_url / github_url</code>	<code>CharField</code>	nullable
<code>profile_summary</code>	<code>TextField</code>	자기소개 요약
<code>skills_json</code>	<code>JSONField</code>	{languages, frameworks, databases, tools}
<code>experience_json</code>	<code>JSONField</code>	[[company, role, period, duties]]
<code>education_json</code>	<code>JSONField</code>	[[school, major, degree, period]]

필드	타입	설명
projects_json	JSONField	프로젝트 목록
skill_tags	JSONField	["python", "django", ...]

3-3. dataset_job_descriptions (구조화 채용공고)

필드	타입	설명
entry_id	OneToOneField	FK→dataset_entries
job_posting_id	ForeignKey	FK→job_postings (nullable)
company_name / job_title	CharField	
location / employment_type / department	CharField	nullable
responsibilities	JSONField	주요 업무 목록
qualifications	JSONField	{required:[], preferred:[]}
benefits	JSONField	복지 목록
required_skill_tags / preferred_skill_tags	JSONField	스킬 태그 배열

3-4. dataset_match_scores (AI 매칭 학습 데이터)

이력서-채용공고 매칭 점수. AI 모델 학습용 레이블 데이터.

필드	타입	설명
resume_id	ForeignKey	FK→dataset_resumes
jd_id	ForeignKey	FK→dataset_job_descriptions
is_positive	BooleanField	매칭 여부 (True=매칭)
rule_score	FloatField	규칙 기반 점수
ai_score	FloatField	AI 모델 점수
final_score	FloatField	최종 가중 점수
score_detail	JSONField	세부 점수 breakdown

유니크: (resume, jd)

3-5. dataset_load_history (적재 이력)

필드	타입	설명
dataset_name	CharField(200)	HuggingFace 데이터셋 경로
status	CharField(10)	running / success / failed
total_rows / loaded_rows / failed_rows	PositiveIntegerField	적재 통계
error_message	TextField	실패 시 오류 메시지
started_at / finished_at	DateTimeField	배치 시간

4. 레이어 3 — 문제·퀴즈 (7개 테이블)

[JobProblems/](#) (30개 직군 × 200문제 = 6,000개)와 [LearningPaths/](#) (클러스터·엣지·메타), 그리고 진단 퀴즈·추천 세션 상태를 저장합니다.

4-1. job_problems (직군 문제)

필드	타입	제약	설명
id	BigAutoField	PK	
original_question_id	PositiveIntegerField	NOT NULL	파일 내 1~200 번호
job_role	CharField(100)	NOT NULL	예) AI Engineer, Backend Engineer
difficulty	CharField(20)	NOT NULL	university_level / junior_level / middle_level / senior_level
question_type	CharField(20)	nullable	definition / application / debugging / design / analysis / coding
category / subcategory	CharField(100)		예) Machine Learning / Evaluation
skills_required	JSONField	NOT NULL	예) ["Metrics", "ROC AUC"]
scenario	TextField	nullable	문제 상황 설명
question	TextField	NOT NULL	문제 본문
choices	JSONField	NOT NULL	객관식 선택지 배열
correct_answer	CharField(300)	NOT NULL	정답 문자열
explanation	TextField	nullable	해설
total_attempts / correct_count	PositiveIntegerField	default=0	풀이 통계 (자동 갱신)
avg_correct_rate	FloatField	nullable	전체 정답률 (%)

유니크: (job_role, original_question_id) 인덱스: idx_jp_job_role idx_jp_difficulty idx_jp_category
idx_jp_qtype

4-2. job_problem_clusters (문제 클러스터)

같은 category/subcategory로 묶인 문제 그룹. 스킬 갭 분석 및 추천에 활용됩니다.

필드	타입	설명
job_role	CharField(100)	
cluster_id	CharField(200)	예) "Deep Learning__Optimization"
category / subcategory	CharField(100)	
size	PositiveSmallIntegerField	클러스터 내 문제 수
question_ids	JSONField	original_question_id 배열
common_skills	JSONField	공통 스킬 태그 배열
skill_weight / scenario_weight	FloatField	LearningPath 생성 파라미터

유니크: (job_role, cluster_id) 인덱스: idx_jpc_job_role idx_jpc_category

4-3. problem_edges (선수과목 관계 그래프)

문제 A → 문제 B: A를 먼저 풀어야 B를 잘 풀 수 있는 방향성 엣지.

필드	타입	설명
job_role	CharField(100)	
source_problem_id	ForeignKey	FK→job_problems (선수 문제)
target_problem_id	ForeignKey	FK→job_problems (후속 문제)
combined_score	FloatField	스킬+시나리오 가중 합산 점수
skill_overlap	FloatField	스킬 겹침 정도 0~1
scenario_similarity	FloatField	시나리오 유사도 0~1
is_prerequisite	BooleanField	True=선수과목 관계

유니크: (source_problem, target_problem) 인덱스: idx_pe_job_role idx_pe_prereq idx_pe_score
idx_pe_target

4-4. learning_path_meta (학습 경로 메타)

직군별 LearningPath 생성 메타 정보. 30개 직군 × 1행.

필드	타입	설명
job_role	CharField(100)	UNIQUE
source_file	CharField(200)	원본 JSON 파일명
total_problems / cluster_count / edge_count	PositiveSmallIntegerField	통계
isolated_problem_count	PositiveSmallIntegerField	독립 문제(클러스터 미소속) 수
difficulty_distribution	JSONField	{university_level:25, junior_level:25, ...}
skill_weight / scenario_weight	FloatField	그래프 생성 파라미터

4-5. job_problem_solve_history (자체 문제 풀이 이력)

외부 플랫폼(solve_history)과 별도로 ELAW 자체 문제 풀이를 기록합니다.

save() 호출 시 job_problems의 total_attempts, correct_count, avg_correct_rate를 자동 갱신합니다.

필드	타입	설명
user_id	ForeignKey	FK→users
problem_id	ForeignKey	FK→job_problems
from_recommendation_id	ForeignKey	FK→problem_recommendations (nullable)
status	CharField(10)	correct / incorrect / skipped
selected_answer	CharField(300)	사용자 선택 답
time_spent_sec	PositiveSmallIntegerField	풀이 소요 시간(초)
solved_at	DateTimeField	auto_now_add

인덱스: `idx_jpsh_user_time` `idx_jpsh_user_status` `idx_jpsh_problem`

4-6. quiz_sessions (진단 퀴즈 세션) 신규

10문제 진단 퀴즈의 진행 상태를 사용자별로 DB에 저장합니다 (쿠키/메모리 불필요 — 서버 재시작에도 세션 유지). 사용자당 1행(OneToOne)이며, 새 퀴즈 시작 시 덮어씁니다.

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	OneToOneField	FK→users (CASCADE), UNIQUE	사용자당 1개 세션

필드	타입	제약	설명
job_role	CharField(100)	NOT NULL	진단 대상 직군
session_data	JSONField	NOT NULL	출제 문제·응답 진행 상태 직렬화
created_at / updated_at	DateTimeField	auto	

4-7. recommend_sessions (추천 세션) 신규

진단 퀴즈 완료 후 산출된 수준 구간(zone)·양상블 가중치·취약 카테고리·추천 결과를 사용자별로 보관합니다. 스터디 모드의 KT 양상블(GKT·SAKT·DKT)이 이 세션을 읽고 갱신합니다.

필드	타입	제약	설명
id	BigAutoField	PK	
user_id	OneToOneField	FK→users (CASCADE), UNIQUE	사용자당 1개 세션
job_role	CharField(100)	NOT NULL	
zone	CharField(20)	NOT NULL	낮음 / 괜찮음 / 높음 (정답률 구간)
weights	JSONField	NOT NULL	{GKT, SAKT, DKT} 양상블 가중치
weak_categories	JSONField	NOT NULL	취약 카테고리 목록
responses	JSONField	NOT NULL	퀴즈 응답 이력
recommendations	JSONField	NOT NULL	추천 문제 목록
created_at / updated_at	DateTimeField	auto	

5. 레이어 4 — 추천·갭·포트폴리오 (4개 테이블)

5-1. skill_gaps (스킬 갭 분석)

사용자 현재 역량(learning_stats)과 채용공고 요구 수준(job_postings)의 차이를 수치화합니다.

필드	타입	설명
user_id	ForeignKey	FK→users
posting_id	ForeignKey	FK→job_postings (nullable)
skill_name	CharField(100)	예) Python, DP, 그래프
skill_category	CharField(30)	language / algo_tag / framework
current_level	FloatField	현재 역량 0~100
required_level	FloatField	공고 요구 수준 0~100
gap_score	FloatField	required - current (음수=충분)
recommended_problem_ids	JSONField	갭을 채울 추천 문제 ID 목록
status	CharField(10)	pending(분석 대기) / active(학습 중) / resolved(해소됨) — 0007 추가
calculation_method	CharField(50)	rule_based / model_based

유니크: (user, posting, skill_name)

5-2. problem_recommendations (문제 추천)

스킬 갭을 채우기 위해 추천된 문제 1건. 추천 이후 풀이 이력과 연결됩니다.

필드	타입	설명
user_id	ForeignKey	FK→users
posting_id	ForeignKey	FK→job_postings (nullable)
skill_gap_id	ForeignKey	FK→skill_gaps (nullable)
problem_id	CharField(100)	플랫폼 내 문제 ID
platform	CharField(20)	elaw(자체 문제) / baekjoon / programmers / leetcode
title / algo_tags / difficulty	CharField/JSONField	문제 메타정보

필드	타입	설명
relevance_score	FloatField	추천 관련성 점수 0~1
reason	TextField	추천 이유 텍스트
status	CharField(10)	pending / solved / skipped
recommended_at / solved_at	DateTimeField	

유니크: (user, platform, problem_id, posting)

5-3. portfolio_snapshots (포트폴리오 버전)

AI가 생성한 포트폴리오의 각 버전. is_final=True인 버전이 사용자에게 표시됩니다.

필드	타입	설명
portfolio_id	ForeignKey	FK→portfolios
user_id	ForeignKey	FK→users
posting_id	ForeignKey	FK→job_postings (nullable)
structured_resume / skill_analysis / problem_history	JSONField	생성 당시 입력 데이터 스냅샷
generated_content	JSONField	섹션별 생성 결과
generation_method	CharField(20)	gemini / model_v1 / model_v2 / manual
generation_prompt	TextField	재현·개선용 프롬프트 보관
version	PositiveSmallIntegerField	버전 번호
is_final	BooleanField	현재 표시 버전 여부
quality_score	FloatField	자동 품질 평가 점수

5-4. portfolio_feedback (RLHF 피드백)

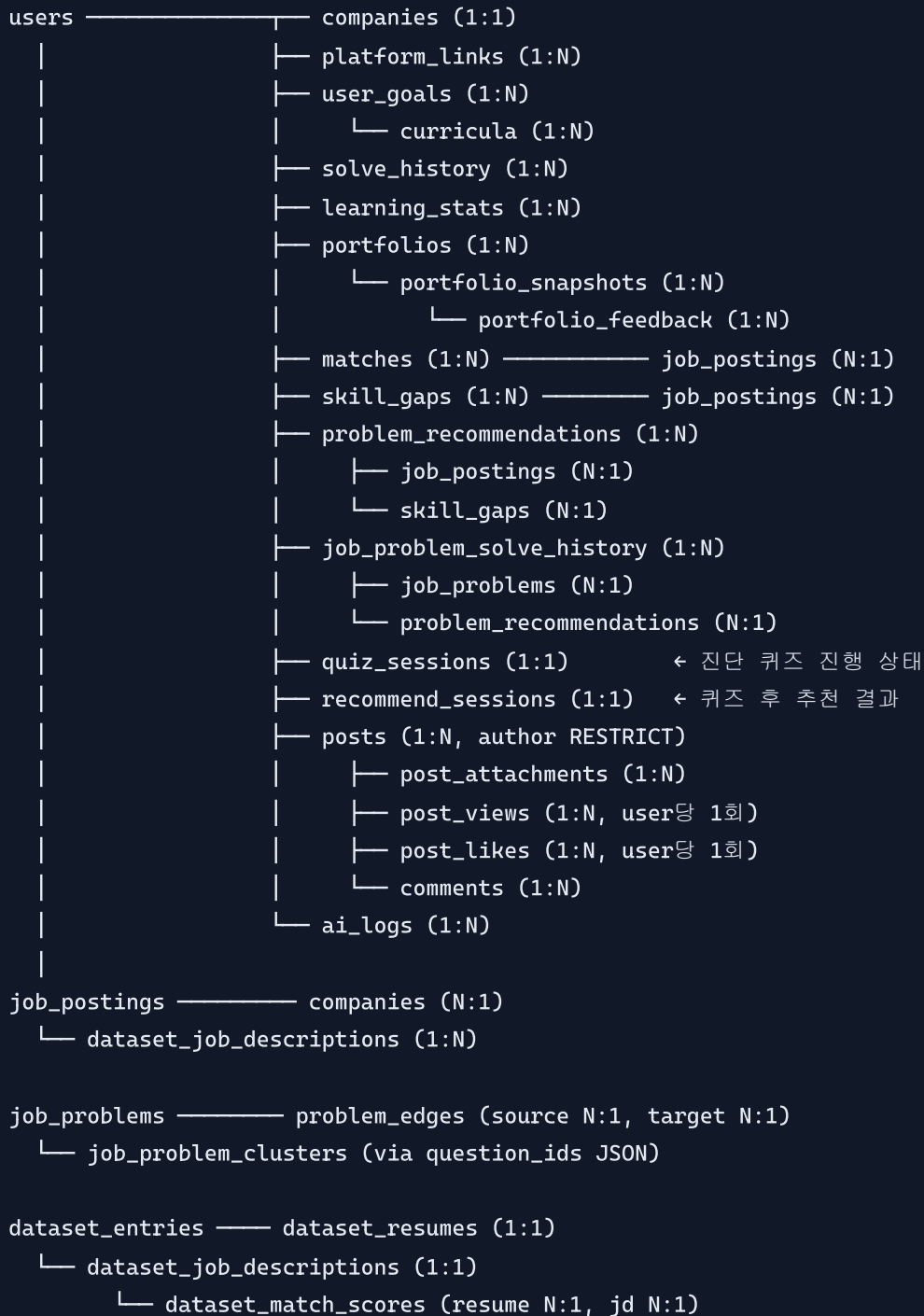
사용자가 포트폴리오 스냅샷에 남긴 피드백. used_for_training=True 데이터로 모델을 재학습합니다.

필드	타입	설명
snapshot_id	ForeignKey	FK→portfolio_snapshots
user_id	ForeignKey	FK→users
rating	IntegerField	1(매우불만족) ~ 5(매우만족)

필드	타입	설명
feedback_text	TextField	자유 텍스트
edited_content	JSONField	사용자 수정 내용 (원본 비교용)
rlhf_labels	JSONField	{overall, relevance, completeness, preferred_sections, edit_distance}
used_for_training	BooleanField	모델 재학습 사용 여부
training_batch_id	CharField(100)	학습 배치 식별자

유니크: (snapshot, user)

6. 테이블 관계도 (ERD 요약)



7. 데이터 흐름

7-1. 문제 데이터 적재 흐름

- 1 JobProblems/ JSON 파일 (30개 직군 × 200문제) → `job_problems` 테이블 6,000건 적재
- ↓
- 2 LearningPaths/ JSON 파일 → `job_problem_clusters` (5,214건), `problem_edges` (15,323건), `learning_path_meta` (30건) 적재
- ↓
- 3 사용자 문제 풀이 → `job_problem_solve_history` 저장 → `job_problems` 통계 자동 갱신

7-1b. 진단 퀴즈 → 추천 흐름

- 1 스터디 모드 진입 → SeedQuiz 10문제 출제 → 진행 상태를 `quiz_sessions` 에 저장 (사용자당 1행)
- ↓
- 2 퀴즈 완료 → 정답률로 수준 구간(zone) 판별 → 구간별 GKT·SAKT·DKT 가중치 산출 → `recommend_sessions` 저장
- ↓
- 3 양상불이 다음 문제 추천 → 풀이 결과 `job_problem_solve_history` 기록 + 세션 온라인 업데이트

7-2. 학습 분석 흐름

- 1 `sync_platforms` → 백준·GitHub API → `solve_history` 동기화
- ↓
- 2 `solve_history` 집계 → `learning_stats` (언어별·태그별 정답률) 갱신
- ↓
- 3 `learning_stats` + `job_postings.required_skills` 비교 → `skill_gaps` 생성
- ↓
- 4 `skill_gaps` + `problem_edges` (선수과목) → `problem_recommendations` 생성

7-3. 포트폴리오 생성 및 RLHF 흐름

- 1 `dataset_resumes` (구조화 이력서) + `skill_gaps` + `solve_history` 요약 → Gemini API 입력
- ↓
- 2 Gemini 응답 → `portfolio_snapshots` 저장 (v1, is_final=True)
- ↓
- 3 사용자 피드백 → `portfolio_feedback` 저장 (rlhf_labels 포함)
- ↓
- 4 `used_for_training=True` 데이터 축적 → 자체 모델 재학습 (model_v1 → model_v2)

7-4. 매칭 흐름

- 1 `dataset_entries` (HuggingFace) → Gemini 파싱 → `dataset_resumes` + `dataset_job_descriptions`
- ↓
- 2 이력서·공고 쌍 매칭 → `dataset_match_scores` (AI 학습 레이블)
- ↓
- 3 실사용자 `portfolios` + 기업 `job_postings` → AI 매칭 → `matches`

8. 마이그레이션 히스토리

#	파일명	적용 일	내용
0001	0001_initial.py	초기	기본 도메인 12개 테이블 생성
0002	0002_user_is_staff_ ...		User 모델에 is_staff, is_superuser 필드 추가
0003	0003_dataset_tables.py		데이터셋 5개 테이블 추가 (수동 작성)
0004	0004_merge_20260421.py		0002:0003 병합 마이그레이션
0005	0005_alter_datasetentry_ ...	2026-04-27	문제 5개 테이블 추가, 데이터셋 필드 수정
0006	0006_portfoliosnapshot_ ...	2026-04-27	추천·갭·포트폴리오 4개 테이블 추가, from_recommendation 필드 복구
0007	0007_add_skillgap_status_problemrec_elaw_platform.py	2026-05	skill_gaps.status 추가, problem_recommendations.platform에 elaw 추가
0008	0008_add_quiz_recommend_sessions.py	2026-05	quiz_sessions-recommend_sessions 2개 테이블 추가 (진단 퀴즈 DB 세션화)
0009	0009_board_comments_likes_views.py	2026-05	comments-post_likes-post_views 추가, posts.like_count-qna 카테고리 추가
0010	0010_post_attachments.py	2026-05	post_attachments(게시글 첨부파일) 추가
0011	0011_alter_jobproblem_question_type.py	2026-05	job_problems.question_type 길이 확장

마이그레이션 전체 적용 완료 (0011까지). `python manage.py showmigrations core` 로 확인 가능.

Supabase 운영 DB 주의: `migrate` 실행 후 반드시 `python scripts/apply_supabase_qls.py` 를 재실행해야 합니다. 새로 생긴 테이블은 RLS 미적용 상태로 PostgREST(anon key)에 노출되기 때문입니다.

9. 관리 커맨드

load_problems — 문제 데이터 적재

```
# 전체 적재
python manage.py load_problems --problems_dir ./DB/JobProblems --paths_dir ./DB/LearningPaths

# 초기화 후 재적재
python manage.py load_problems --problems_dir ./DB/JobProblems --paths_dir ./DB/LearningPaths --reset

# 특정 직군만
python manage.py load_problems --problems_dir ./DB/JobProblems --job_role "AI Engineer"
```

load_dataset — HuggingFace 데이터셋 적재

```
# 전체 2,640건
python manage.py load_dataset

# 처음 100건만
python manage.py load_dataset --limit 100

# 적재 후 Gemini로 파싱 (최대 50건)
python manage.py load_dataset --parse

# 특정 기업 · 경력 필터
python manage.py load_dataset --company Google --level junior
```

seed_all — 테스트 시드 데이터

```
python manage.py seed_all
# 학생 10명, 기업 3개, 채용공고 6개, 플이이력 · 통계 · 매칭 등 전체 생성
```

sync_platforms — 외부 플랫폼 ETL 동기화

```
# 전체 활성 학생 사용자 동기화
python manage.py sync_platforms --all

# 특정 사용자 · 플랫폼만
python manage.py sync_platforms --user=minjun.kim@elaw.kr --platform=baekjoon
```

Supabase 운영 스크립트 (저장소 루트 scripts/)

```
# 대량 적재 - loaddata 대신 사용 (bulk_create, 약 30k 객체 ~6초; loaddata는 WAN에서 ~90분)
python scripts/fast_loaddata.py backend/backup_sqlite.json

# RLS 정책 (재)적용 - migrate 후 필수
python scripts/apply_supabase_qls.py

# 전체 public 테이블 드롭 (기본은 dry-run, --yes로 실행)
python scripts/supabase_reset.py --yes
```

모든 커맨드는 가상환경 활성화 후 실행해야 하며, Windows에서는 `PYTHONUTF8=1` 환경변수가 필수입니다 (cp949 인코딩으로 한글 출력·dumpdata가 깨짐).

10. DB 조회 API

모든 엔드포인트는 JWT 인증이 필요합니다 (`Authorization: Bearer <token>`).

메서드	URL	설명
GET	<code>/api/db/users/</code>	전체 사용자 목록
GET	<code>/api/db/companies/</code>	기업 목록
GET	<code>/api/db/platform-links/</code>	플랫폼 연동 목록
GET	<code>/api/db/user-goals/</code>	학습 목표 목록
GET	<code>/api/db/curricula/</code>	커리큘럼 목록
GET	<code>/api/db/solve-history/</code>	외부 플랫폼 풀이 이력
GET	<code>/api/db/learning-stats/</code>	학습 통계
GET	<code>/api/db/portfolios/</code>	포트폴리오 목록
GET	<code>/api/db/job-postings/</code>	채용공고 목록
GET	<code>/api/db/matches/</code>	매칭 결과 목록
GET	<code>/api/db/posts/</code>	게시글 목록
GET	<code>/api/db/ai-logs/</code>	AI 호출 로그
GET	<code>/api/db/summary/</code>	주요 테이블 건수 요약

11. 환경 설정

운영 (Supabase PostgreSQL — 현재 기본)

`backend/.env` 의 `DB_ENGINE=postgresql` 이 설정되면 Supabase Session pooler로 직접 접속합니다. 서울 리전, 전체 데이터 단일 저장소입니다.

```
# backend/.env
DB_ENGINE=postgresql
DB_NAME=postgres
DB_USER=postgres.<project-ref>
DB_PASSWORD=*****
DB_HOST=aws-0-ap-northeast-2.pooler.supabase.com # Session pooler
DB_PORT=5432
```

로컬 풀백 (SQLite)

`DB_ENGINE` 을 주석 처리하면 `db.sqlite3` 로 풀백합니다. MySQL 분기 코드도 존재하지만 사용하지 않습니다.

RLS (Row-Level Security) — Supabase 보안 정책

Supabase는 PostgREST를 통해 anon key로 테이블이 노출될 수 있으므로, 모든 public 테이블에 RLS를 적용합니다 (`DB/sql/05_rls_policies.sql` / `scripts/apply_supabase_rls.py`).

- 모든 테이블: RLS 활성화 + anon/authenticated의 INSERT-UPDATE-DELETE 권한 회수
- 공개 읽기(anon SELECT) 7개 테이블만 허용 — 랜딩 페이지가 supabase-js로 직접 읽는 테이블: `job_problems` , `job_problem_clusters` , `problem_edges` , `learning_path_meta` , `core_company` (승인된 기업만), `core_jobposting` (활성 공고만), `core_post`
- 사용자·세션·dataset 테이블은 정책 없음 → PostgREST 접근 차단
- Django는 테이블 소유자(postgres)로 접속하므로 RLS의 영향을 받지 않음
- migrate**로 새 테이블이 생기면 반드시 **재실행** (idempotent)

필수 환경변수 (.env)

변수명	용도	필수
<code>DB_ENGINE</code> / <code>DB_NAME</code> / <code>DB_USER</code> / <code>DB_PASSWORD</code> / <code>DB_HOST</code> / <code>DB_PORT</code>	Supabase PostgreSQL 연결 (Session pooler)	운영 필수
<code>GEMINI_API_KEY</code>	커리큘럼(use_ai=true)-데이터셋 파싱 AI	운영 필수
<code>GITHUB_TOKEN</code>	GitHub API ETL rate limit 상향 (60→5,000 회/시간)	선택

변수명	용도	필수
DJANGO_SECRET_KEY	Django 보안 키	운영 필수
DJANGO_ALLOWED_HOSTS	.trycloudflare.com 포함 필요 (터널 접속)	운영 필수

운영 주의: ① Windows에서 manage.py 실행 시 PYTHONUTF8=1 필수. ② Supabase에 loaddata 금지 (행 단위 WAN 왕복 ≈ 90분) — scripts/fast_loaddata.py 사용. ③ DEBUG=False-CORS 제한은 배포 전 점검 항목.

12. 현재 데이터 현황 (2026-06-05, Supabase 운영 DB 실측)

6,000 직군 문제 job_problems	5,214 문제 클러스터 job_problem_clusters	15,323 선수과목 엣지 problem_edges
30 학습경로 메타 learning_path_meta	1,320 데이터셋 항목 dataset_entries	1,346 채용공고 core_jobposting
30 사용자 core_user	14 기업 core_company	227 풀이 이력 core_solvehistory
182 학습 통계 core_learningstats	20 플랫폼 연동 core_platformlink	23 포트폴리오 core_portfolio
46 매칭 core_match	32 목표 / 커리큘럼 core_usergoal · core_curriculum	5 게시글 core_post
3 / 2 퀴즈·추천 세션 quiz_sessions · recommend_sessions		

비고: 사용자·풀이 이력·학습 통계·플랫폼 연동의 상당수는 `seed_all` 이 생성한 테스트용 시드 데이터입니다 (시드 학생 10명, 가상 백준/GitHub 핸들). 실사용자 계정의 ETL 동기화로 교체하는 작업이 진행 중이며, `dataset_resumes` / `portfolio_snapshots` / `skill_gaps` 등 AI 학습용 테이블은 현재 0건(향후 확장 영역)입니다.

